

Base Analysis Complete Guide

Candidate R1 - Complete Question-Driven and Example-Enriched Guide

Guida completa del contratto operativo BA con question-driven discrimination e worked examples source-grounded

Questa guida consolida direttamente il core semantico/sintattico della Core Guide companion e integra worked examples source-grounded, full question-driven discrimination, pairwise boundaries e matrici finali. I checkpoint intermedi usati durante la costruzione non sono richiesti come artifact di repository in R42.

Stato	CANDIDATE / NON-NORMATIVE / QUESTION-COMPLETE / EXAMPLE-ENRICHED / REVIEW REQUIRED
Scopo di questa revisione	Completezza semantica, sintattica, question-driven e pedagogica; promotion ancora subordinata a review cumulativa rispetto alla R3 current.
Authority BA corrente	DDTA_BASE_ANALYSIS_OPERATIONAL_GUIDE_R3 . tex + BA0 R1 / BA1 R1 / BA2 R3 / BA3 R1 / BA4 R1 / BA5 R1.
Operator basis corrente	14 top-level operators. Nessun quindicesimo operatore e' ammesso.
Repository baseline di partenza	bea1e6567d3f861294378b201648b0853b3c8a43
Core companion	DDTA_BASE_ANALYSIS_CORE_GUIDE_R1_CANDIDATE_R1: core semantico/sintattico preservato; questa guida aggiunge esempi e question-driven discrimination.
Layer esempi	Preservato: Facial Access, DermaTriage, boundary examples e controesempi.
Question layer	Aggiunto: question set Q00-Q52, pairwise discriminators, stopping questions e quick decision matrix.
Data	12 settembre 2026

Promotion stop esplicito

Questa Complete Guide candidate non viene promossa automaticamente. Gli esempi aumentano la completezza pedagogica ma non trasformano evidence in authority. La promotion richiede ancora la verifica cumulativa sezione-per-sezione rispetto a R3 e la chiusura esplicita delle questioni che restano OPEN.

Indice

1	Scopo, lineage e regola di costruzione	7
1.1	Legenda epistemica	7
2	Che cos'è la Base Analysis	7
3	Contratto BA0-BA5 e gate BA6	8
4	Procedura operativa end-to-end	8
5	BAReferent	9
5.1	Quando creare un BAREferent	9
5.2	Quando NON creare un BAREferent	9
6	BAProposition	10
6.1	Invarianti	10
7	Criterio di dettaglio, minimalità e stopping rule	10
8	Operator selection procedure	10
9	Question-driven discrimination: full question set	11
9.1	Q00-Q06 - authority, scope e semantic atom	11
9.2	Q07-Q12 - BAREferent o BAProposition?	12
9.3	Q13-Q24 - azione, comportamento e risultato	12
9.4	Q25-Q35 - relazioni tra significati	13
9.5	Q36-Q43 - constraint, classification e decision semantics	13
9.6	Q44-Q52 - strutture locali, pipeline, candidate e stopping	14
9.7	Pairwise discriminators: le coppie che più facilmente si confondono	14
9.8	Decision path compatto	16
10	RoleKey: significato e disciplina	16
11	Matrice normativa delle cardinalità	17
12	Operator: transfer	18
12.1	Definizione	18
12.2	Domanda diagnostica	18
12.3	Firma canonica	18
12.4	Campi	19

12.5	Invarianti e confini	19
12.6	Forbidden inference	19
12.7	Worked examples	19
13	Operator: produce	20
13.1	Definizione	20
13.2	Domanda diagnostica	20
13.3	Firma canonica	20
13.4	Campi	21
13.5	Invarianti e confini	21
13.6	Forbidden inference	21
13.7	Worked examples	21
14	Operator: create	22
14.1	Definizione	22
14.2	Domanda diagnostica	22
14.3	Firma canonica	22
14.4	Campi	22
14.5	Invarianti e confini	22
14.6	Forbidden inference	22
14.7	Worked examples e negative control	23
15	Operator: observe	23
15.1	Definizione	23
15.2	Domanda diagnostica	23
15.3	Firma canonica	23
15.4	Campi	24
15.5	Invarianti e confini	24
15.6	Open delta	24
15.7	Worked examples e confine con selection	24
16	Operator: transition	25
16.1	Definizione	25
16.2	Domanda diagnostica	25
16.3	Firma canonica	25
16.4	Campi	25
16.5	Invarianti e confini	25
16.6	Open delta	25

16.7	Worked examples e boundary control	26
17	Operator: correlate	26
17.1	Definizione	26
17.2	Domanda diagnostica	26
17.3	Firma canonica	26
17.4	Campi	27
17.5	Invarianti e confini	27
17.6	Worked examples	27
18	Operator: reference	27
18.1	Definizione	27
18.2	Domanda diagnostica	28
18.3	Firma canonica	28
18.4	Campi	28
18.5	Invarianti e confini	28
18.6	Worked examples	28
19	Operator: dependOn	29
19.1	Definizione	29
19.2	Domanda diagnostica	29
19.3	Firma canonica	29
19.4	Campi	29
19.5	Invarianti e confini	29
19.6	Worked examples	29
20	Operator: consumeService	30
20.1	Definizione	30
20.2	Domanda diagnostica	30
20.3	Firma canonica	30
20.4	Campi	30
20.5	Invarianti e confini	30
20.6	Worked examples	31
21	Operator: realize	31
21.1	Definizione	31
21.2	Domanda diagnostica	31
21.3	Firma canonica	31

21.4	Campi	32
21.5	Invarianti e confini	32
21.6	Worked examples	32
22	Operator: assignResponsibility	32
22.1	Definizione	32
22.2	Domanda diagnostica	33
22.3	Firma canonica	33
22.4	Campi	33
22.5	Invarianti e confini	33
22.6	Worked examples e responsibility boundary	33
23	Operator: constrain	34
23.1	Definizione	34
23.2	Domanda diagnostica	34
23.3	Firma canonica	34
23.4	Campi	34
23.5	Invarianti e confini	34
23.6	Worked examples	35
24	Operator: classify	35
24.1	Definizione	35
24.2	Domanda diagnostica	35
24.3	Firma canonica	35
24.4	Campi	35
24.5	Invarianti e confini	36
24.6	Worked examples	36
25	Operator: decisionRule	36
25.1	Definizione	36
25.2	Domanda diagnostica	36
25.3	Firma canonica	37
25.4	Campi top-level	37
25.5	Invarianti e confini	37
25.6	Worked examples	37
26	Condition language di decisionRule	38
26.1	comparison	38

26.2	satisfies	39
26.3	allOf	39
26.4	anyOf	39
26.5	not	39
26.6	resultAssignment	39
26.7	Worked examples della condition language	40
27	Struttura locale candidate: selection	40
27.1	Working structure	41
27.2	Distinzioni obbligatorie	41
27.3	Worked comparison: tre forme diverse di selection in DermaTriage	41
28	Pressure aperta: pipeline composition e order	42
28.1	Domande da porre	42
28.2	Worked boundary: DermaTriage DEC-12	42
29	Pressure e strutture trasversali ancora aperte	43
30	Candidate construct R25 non ammessi	43
30.1	CC-01 consumeData	43
30.2	CC-02 provideService	43
30.3	CC-03 storedIn	44
30.4	CC-04 initiate	44
30.4.1	Working minimum signature	44
30.4.2	Semantica minima	44
30.4.3	Non implica	44
30.5	Worked candidate examples R25	45
31	Working hypothesis respinte	45
32	BA3 - provenance, origin state e revalidation	45
32.1	Origin states	46
32.2	Tre responsabilita' distinte	46
32.3	Derived semantics	46
32.4	Change e revalidation	46
33	Diagnostics e unresolved semantics	46
33.1	Quando preferire una diagnosis a una proposition	47
34	BA5 - authoring canonico	47

35 BA4 - projection boundary	47
35.1 Coverage modes	48
35.2 Projection non autorizza inference upstream	48
36 Regression e acceptance della Base Analysis	48
36.1 Checklist pre-acceptance	48
37 Open register R25 da preservare nella prossima revisione	49
38 Cosa questa revisione non chiude	49
39 Gate di review della candidate example-enriched	50
A Quick syntax sheet - top-level operators	50
B Quick syntax sheet - condition language	51
C Quick status sheet - non-current constructs	52
D Matrice degli accoppiamenti semantici ricorrenti	53
E Matrice rapida: quale operatore provo per primo?	56
F Quick question matrix - dalla domanda al costruito	56
G Provenance degli esempi principali	58

1 Scopo, lineage e regola di costruzione

Questa guida separa deliberatamente due obiettivi che nelle revisioni precedenti erano intrecciati:

1. **completezza semantica e sintattica:** ogni costruito deve avere significato, firma, ruoli, cardinalita', confini e stato;
2. **completezza pedagogica:** ogni costruito deve essere accompagnato da esempi source-grounded e controesempi.

La Complete Guide affronta tre obiettivi: preserva il contratto semantico/sintattico della Core Guide companion, integra gli esempi source-grounded Facial Access e DermaTriage e applica un question set esplicito per discriminare i costrutti senza modificarne silenziosamente lo status.

Regola cumulativa

Una candidate successore non puo' perdere una distinzione stabile della R3 per essere piu' breve. Ogni contenuto R3 viene quindi classificato come preservato, espanso, qualificato oppure accompagnato da evidence esplicitamente non normativa. Nessuna evidence R25 diventa authority soltanto perche' e' inclusa in questa guida.

1.1 Legenda epistemica

Etichetta	Significato
CURRENT / PRESERVED	Regola gia' authority corrente in R3/BA0-BA5.
R25 EVIDENCE	Evidenza positiva o pressure confermata dal secondo ciclo DermaTriage.
CANDIDATE / NOT ADMITTED	Costrutto sopravvissuto a test parziali ma non formalmente ammesso.
REJECTED	Working hypothesis respinta nello scope corrente.
OPEN / COUNTEREXAMPLE-GATED	Questione semanticamente reale ancora non chiusa.
EXAMPLE-ENRICHED	La semantica e' accompagnata da esempi source-grounded, boundary examples e negative control quando necessario.

2 Che cos'e' la Base Analysis

CURRENT / PRESERVED

Per una baseline documentale governata, la *Base Analysis* e' la rappresentazione analitica condivisa e methodology-neutral del significato di progetto necessario ai consumer downstream DDTA.

```
governed project documentation
  = project authority

accepted Base Analysis
  = shared analytical baseline

projection / threat analysis / tests
  = downstream consumers
```

La BA puo' normalizzare, rendere riusabile e rendere interrogabile un significato gia' governato. Non puo'

completare silenziosamente la documentazione, inventare un commitment, scegliere una interpretazione non supportata o trasformare una domanda downstream in project truth.

Authority direction

Documentazione governata → BA → projection/analysis. Il feedback puo' risalire come richiesta di review, ma una modifica di project truth avviene soltanto tramite una nuova decisione documentale governata e successiva revalidation della BA.

3 Contratto BA0–BA5 e gate BA6

CURRENT / PRESERVED

Layer	Responsabilita'
BA0	authority boundary, minimum sufficient analysis, unresolved state, feedback direction e anti-inference.
BA1	due sole famiglie first-class: <i>BAReferent</i> e <i>BAProposition</i> .
BA2 R3	proposition semantics, top-level operators, operator-scoped roleKey, cardinalita' e strutture locali.
BA3	provenance, origin state, derivation, review/freshness, continuity e revalidation.
BA4	projection, traceability, interpretation e coverage downstream.
BA5	canonical semantic registries, domain-scoped resolution e controlled authoring.
BA6	integrated validation gate. Non viene dichiarato chiuso da questa guida.

BA2 authority corrente

Le firme di operatori, roleKey e cardinalita' normative derivano da BA2_RELATION_ACTION_VOCABULARY_R3. Snapshot storici incorporati in altri contratti non sostituiscono BA2 R3.

4 Procedura operativa end-to-end

CURRENT / PRESERVED

1. Pin di baseline, authority e source revision.
2. Lettura source-first del governed source set.
3. Identificazione del minimum shared meaning necessario.
4. Identificazione dei significati che richiedono identita' riusabile.
5. Creazione dei BAReferent necessari.
6. Formulazione delle BAProposition minime usando BA2 R3.
7. Verifica di firma, roleKey, cardinalita' e scope delle proposition.
8. Collegamento BA3 a source evidence, origin state e revalidation context.

9. Registrazione esplicita di ambiguity, conflict, diagnosis e NOT SPECIFIED.
10. Minimum-BA acceptance per lo scope dichiarato.
11. Regression completa verso le fonti e forbidden-inference review.
12. Method-pressure review soltanto su counterexample concreti.
13. Materializzazione della accepted case BA.
14. Projection/downstream consumption secondo BA4.
15. Governed change e BA3 revalidation quando necessario.
16. Integrated BA6 verdict soltanto dopo l'intero protocollo previsto.

5 BAREferent

CURRENT / PRESERVED

Un *BAREferent* e' un significato di progetto methodology-neutral che riceve identita' analitica indipendente quando tale identita' e' materialmente necessaria.

5.1 Quando creare un BAREferent

Creare un BAREferent quando il significato deve almeno una volta essere:

- riusato da piu' proposition;
- qualificato o vincolato;
- correlato con un altro significato;
- referenziato come target distinto;
- confrontato tra baseline;
- target di provenance/revalidation indipendente;
- oggetto di una domanda downstream materialmente distinta.

5.2 Quando NON creare un BAREferent

Non creare un referent soltanto perche' compare un sostantivo nel testo, perche' un tool desidera un nodo, perche' ogni verbo dovrebbe avere un oggetto o perche' una projection grafica risulterebbe piu' comoda.

Identity test

Se il significato scompare come identita' autonoma senza perdere alcuna distinzione governata, alcuna possibilita' di qualificazione o alcuna tracciabilita' necessaria, probabilmente non richiede un BAREferent separato.

6 BAProposition

CURRENT / PRESERVED

Una *BAProposition* e' una asserzione analitica formalizzata su uno o piu' *BAReferent*. La proposition applica un operatore e assegna *BAReferent* ai *roleKey* ammessi da quell'operatore.

```
BAProposition
  operator -> canonical BA2 operator key
  roles    -> operator-scoped role assignments
  local    -> only operator-local structures when admitted
```

6.1 Invarianti

- una proposition non e' il referent che describe;
- *roleKey* sono scoped all'operatore, non proprieta' universali del referent;
- un ruolo obbligatorio mancante non viene riempito per default;
- un ruolo opzionale si usa soltanto se il significato e' governato;
- una proposition non deve contenere semantica non ammessa dalla propria firma;
- piu' proposition possono essere necessarie per preservare distinzioni diverse contenute nella stessa clausola sorgente.

7 Criterio di dettaglio, minimalita' e stopping rule

CURRENT / PRESERVED

```
more detail
  only if needed to:
    preserve a governed distinction
    answer a material downstream question
    expose a missing governed fact
```

Stopping test

Abbiamo abbastanza semantica per formulare la domanda rilevante e distinguere evidence supportata, contraddetta o NOT SPECIFIED? Se SI, fermarsi. Se NO, identificare la smallest missing distinction o produrre una diagnosis. Non inventare il dettaglio mancante.

8 Operator selection procedure

CURRENT / PRESERVED

La scelta dell'operatore avviene dopo la ricostruzione del significato, non prima.

1. isolare la smallest governed semantic fact;
2. identificare i *BAReferent* necessari;
3. formulare le candidate interpretations materialmente differenti;

4. provare prima gli operatori esistenti;
5. verificare firma e roleKey senza adattamenti opportunistici;
6. applicare delete test e merge-first test;
7. verificare i confini con gli operatori semanticamente vicini;
8. se nessun costrutto preserva il significato, registrare pressure/diagnosis invece di inventare un nuovo operatore locale;
9. un nuovo operatore richiede formal admission separata.

Verb-to-operator mapping

Il verbo naturale nella documentazione non determina automaticamente l'operatore BA. Parole come "acquire", "invoke", "store", "select", "execute", "trigger" e "expose" possono descrivere semantiche differenti a seconda del significato governato.

9 Question-driven discrimination: full question set

CURRENT / PRESERVED

R25 EVIDENCE

Questa sezione rende esplicita la parte interrogativa della procedura. Le domande non sono un questionario documentale e non creano project truth: servono a discriminare rappresentazioni BA candidate dopo che la source authority e' stata fissata. Una risposta "non so" o "la source non lo dice" e' un risultato valido e puo' portare a NOT SPECIFIED, diagnosis o pressure invece che a una proposition artificiale.

Regola di uso del question set

Non iniziare da "quale operatore assomiglia al verbo?". Iniziare da: **quale distinzione semantica e' governata?** Applicare le domande nell'ordine minimo necessario e fermarsi appena una rappresentazione preserva il significato senza aggiungere fatti.

9.1 Q00–Q06 - authority, scope e semantic atom

ID	Domanda guida
Q00	Quale documento/baseline ha authority per il significato che sto analizzando?
Q01	La frase e' project meaning governato, evidence tecnica, commento di review, esempio o inference downstream?
Q02	Qual e' la smallest governed semantic fact che resterebbe vera anche cambiando forma del testo?
Q03	Quali parti della frase sono esplicitamente governate e quali sono soltanto implicite o plausibili?
Q04	Esistono due letture materialmente differenti compatibili con la source? Se SI, qual e' la smallest proposition che le separa?
Q05	La distinzione richiesta e' nello scope della BA corrente o posso fermarmi senza perderla?
Q06	Se elimino il dettaglio candidato, perdo una distinzione governata o una domanda downstream materialmente necessaria? Se NO, STOP.

9.2 Q07–Q12 - BAReferent o BAProposition?

ID	Domanda guida
Q07	Questo significato deve essere riusato, qualificato, correlato, vincolato o tracciato indipendentemente? Se SI, e' candidato BAReferent.
Q08	La source governa una identita' distinguibile oppure soltanto una proprieta'/ruolo locale dentro una proposition?
Q09	Sto creando un BAReferent solo per trasformare ogni verbo o frase in un nodo? Se SI, fermarsi.
Q10	La proposition candidata sta facendo da proxy per un comportamento/oggetto che deve avere identity indipendente? Se SI, creare il BAReferent necessario e mantenere separata la proposition.
Q11	La stessa identity compare in proposition diverse con roleKey differenti? Se SI, non trasformare il roleKey in una classificazione universale del referent.
Q12	La source governa davvero una asserzione tra referent oppure sto soltanto elencando entita' presenti nel testo?

9.3 Q13–Q24 - azione, comportamento e risultato

ID	Domanda guida e instradamento
Q13	Un actor/capability rende disponibile un risultato governato? Provare produce.
Q14	L'azione stabilisce un nuovo item/event con nuova identity/occurrence project-semantic? Provare create; non basta la parola "create".
Q15	Un actor legge/ispeziona un significato gia' esistente senza asserire creazione o state change? Provare observe.
Q16	La stessa identity cambia stato/lifecycle e la source governa almeno il nuovo stato? Provare transition.
Q17	Un content viene realmente conveyed da una source a una destination? Provare transfer.
Q18	La source governa soltanto che una informazione e' usata come input da un'altra operazione? Rappresentarla come input nella proposition pertinente; non introdurre automaticamente consumeData.
Q19	L'elemento chiamato "output" e' un nuovo item, un result reso disponibile, un content trasferito o solo uno state/value? Distinguere prima di scegliere create/produce/transfer/transition.
Q20	"Acquire" governa creazione, osservazione, ricezione/transfer o una combinazione? Se piu' letture restano materialmente possibili, mantenere diagnosis invece di scegliere.
Q21	"Store/persist" governa una nuova identity, uno state/lifecycle, un placement/association o soltanto implementation evidence? Non introdurre un generico persist.
Q22	"Execute/perform" aggiunge una distinzione irriducibile rispetto a produce/create/observe/transition/decisionRule/realize? Se non dimostrato, mantenere PR-01 pressure invece di creare perform.
Q23	La source governa che A avvia B ma non ne governa esito/completamento? Questo e' evidence per initiate candidate, non per un current operator.

ID	Domanda guida e instradamento
Q24	La frase implica successo/completamento solo linguisticamente? Non aggiungerlo se la source non lo governa.

9.4 Q25–Q35 - relazioni tra significati

ID	Domanda guida e instradamento
Q25	Due o piu' elementi devono appartenere allo <i>stesso</i> request/case/evaluation/i-identity context? Provare <i>correlate</i> .
Q26	A deve soltanto identificare/richiamare B senza same-context, prerequisite o ownership semantics? Provare <i>reference</i> .
Q27	A richiede B come prerequisite reale per esistere, funzionare o completarsi? Provare <i>dependOn</i> .
Q28	La source governa solo ordine temporale "A prima di B"? Questo NON basta per <i>dependOn</i> .
Q29	Un consumer usa effettivamente un service/capability separato? Provare <i>consumeService</i> .
Q30	Il fatto che esista un endpoint, API, componente o tecnologia dimostra consumo di servizio? NO: serve semantica di uso della capability/service.
Q31	La source identifica chi fornisce il service consumato? Solo allora valorizzare il roleKey opzionale provider.
Q32	Un elemento concreto materializza un significato astratto gia' governato indipendentemente? Provare <i>realize</i> .
Q33	Sto usando <i>realize</i> solo per dire "X esegue Y"? Se SI, fermarsi: <i>realization</i> richiede la distinzione abstract/concrete.
Q34	La source assegna esplicitamente responsibility/authority a una party rispetto a scope e kind? Provare <i>assignResponsibility</i> .
Q35	Sto confondendo provider/performer/producer/consumer con responsible party? Questi ruoli non implicano automaticamente responsibility/authority.

9.5 Q36–Q43 - constraint, classification e decision semantics

ID	Domanda guida e instradamento
Q36	La source limita un target con un valore, dominio o restriction riusabile/queryable? Provare <i>constrain</i> .
Q37	Il valore e' invece parte locale della condition di una decisione? Preferire la condition language di <i>decisionRule</i> ; non duplicare con <i>constrain</i> senza bisogno.
Q38	La source assegna un semantic kind methodology-neutral a un referent? Provare <i>classify</i> .
Q39	Sto usando <i>classify</i> come fallback per descrivere qualsiasi proprieta' o stato? Se SI, fermarsi.
Q40	La source governa un mapping operativo condizioni/input → risultato? Provare <i>decisionRule</i> .
Q41	Esiste soltanto una azione condizionata senza mapping governato del risultato? Non promuoverla automaticamente a <i>decisionRule</i> ; verificare PR-10/initiate+condition boundary.

ID	Domanda guida e instradamento
Q42	La source governa una condition astratta “X soddisfa Y” senza property/value model interno? Usare <code>satisfies</code> , non inventare un boolean.
Q43	La source governa property e value confrontabili? Usare <code>comparison</code> ; se il confronto e' ordinato, preservare la pressure su <code><</code> , <code>></code> , <code>≤</code> , <code>≥</code> senza inventare vocabulary normativa.

9.6 Q44–Q52 - strutture locali, pipeline, candidate e stopping

ID	Domanda guida e instradamento
Q44	Sto scegliendo uno o piu' risultati da una candidate population secondo un criterio/basis? Provare la <code>local selection</code> structure.
Q45	Sto soltanto ordinando tutti gli elementi? <code>Ranking</code> non e' automaticamente <code>selection</code> .
Q46	Sto soltanto imponendo un massimo numerico? <code>Bound</code> non e' automaticamente <code>selection</code> .
Q47	La source governa <code>top-K</code> , <code>recent-N</code> o <code>best-by-score</code> ? Identificare separatamente <code>candidatePopulation</code> , <code>selectionBasis</code> , <code>selectedResult</code> , eventuale <code>bound</code> e <code>orderingBasis</code> .
Q48	La source governa membership in una pipeline, <code>ordering</code> , <code>prerequisite</code> , <code>transfer</code> o <code>initiation</code> ? Separare queste domande; “pipeline” non e' un operatore corrente.
Q49	Un candidate construct R25 (<code>provideService</code> , <code>storedIn</code> , <code>initiate</code>) sembra adattarsi? Marcarlo candidate e verificare prima se una composizione degli operatori correnti preserva il significato.
Q50	Un'ipotesi gia' respinta (<code>generic consume</code> , <code>modify</code> , <code>persist</code> , <code>expose</code> , <code>trigger</code>) sta riapparendo sotto altro nome? Richiedere nuovo <code>counterexample</code> , non riaprirla per preferenza.
Q51	Nessun operatore/struttura preserva la distinzione senza inventare ruoli o fatti? Registrare pressure/diagnosis; non estendere localmente BA2.
Q52	Una volta rappresentata la distinction minima, ulteriori dettagli cambiano una domanda materiale? Se NO, STOP.

9.7 Pairwise discriminators: le coppie che piu' facilmente si confondono

Coppia / famiglia	Domanda discriminante
<code>produce</code> vs <code>create</code>	Dopo l'azione esiste una nuova identity/occurrence che prima non esisteva? → <code>create</code> . Oppure l'actor rende disponibile un result governato? → <code>produce</code> .
<code>produce</code> vs <code>transfer</code>	Il significato centrale e' generare/rendere disponibile il result o spostare content da source a destination? Le due proposition possono coesistere ma non sono equivalenti.
<code>create</code> vs <code>transition</code>	Nasce una nuova identity oppure la stessa identity cambia stato?
<code>observe</code> vs <code>transfer</code>	La source governa una lettura/ispezione oppure una conveyance? Ricevere content non prova automaticamente <code>observe</code> .

Coppia / famiglia	Domanda discriminante
observe vs selection	L'operazione legge item già determinati oppure decide quali item appartengono al result set? Retrieval e selection possono comporsi.
transition vs property update	La source governa un vero lifecycle/state della stessa identity o solo un valore/proprietà modificato?
correlate vs reference	Serve same-context identity oppure basta un riferimento direzionale?
reference vs dependOn	A deve soltanto richiamare B oppure B è un prerequisite di A?
dependOn vs ordering	B è necessario oppure viene soltanto prima? Temporal order da solo non è dependency.
consumeService vs transfer	A usa la capability/service B oppure content X passa tra endpoint? Possono coesistere sullo stesso boundary.
consumeService vs provider relation	Il current operator descrive il consumo. Il fatto che qualcuno fornisca il servizio non crea automaticamente una proposition provideService; CC-02 resta candidate.
realize vs execution	Esistono davvero abstract meaning e concrete realization distinti e governati, oppure si sta solo dicendo chi esegue qualcosa?
assignResponsibility vs actor/provider	La source assegna responsibility/authority o identifica solo chi agisce/fornisce/consuma?
constrain vs classify	Si limita un target a un valore/dominio oppure si assegna un semantic kind?
constrain vs decisionRule	È una restriction riusabile sul target o una condition che partecipa a un mapping verso result?
classify vs transition	Si assegna una categoria semanticamente stabile o si descrive un cambiamento di stato della stessa identity?
decisionRule vs conditional action	La source governa il result mapping oppure soltanto quando un'azione può/deve essere avviata?
allOf vs multiple propositions	La conjunction appartiene alla stessa decisionCondition oppure sono asserzioni indipendenti che devono restare proposition separate?
not vs non-sufficiency	"A non è sufficiente per B" NON significa not (B). La prima è una boundary/implication statement.
selection vs ranking	Si sceglie un subset/result oppure si definisce solo un ordine?
selection vs bound	Il numero massimo è solo un vincolo oppure parte di un meccanismo di selezione governato?
pipeline order vs dependOn	Lo stadio precedente è un prerequisite semantico o è soltanto precedente nella sequenza?
pipeline order vs transfer	L'ordine degli stadi non prova che un content venga trasferito; verificare source, destination e content.
API/client family	Chiedere separatamente: usa un service? avvia un'operazione? trasferisce content? mantiene correlation? è soggetto a constraint? Non esiste mapping "API = operatore X".
producer/consumer family	Modellare produce sul producer; il result diventa input nella proposition che descrive l'azione del consumer quando la source governa quell'uso. Non introdurre generic consumeData.

9.8 Decision path compatto

SOURCE AUTHORITY

```

|
v
smallest governed semantic fact
|
+-- reusable independent identity? -----> BAREferent
|
'-- assertion between meanings? -----> BAProposition
|
+-----+
|
+-- result made available? -----> produce
+-- new identity/occurrence? -----> create
+-- read/inspect existing meaning? --> observe
+-- same identity changes state? ----> transition
+-- convey content A -> B? -----> transfer
|
+-- same-context binding? -----> correlate
+-- directional reference? -----> reference
+-- prerequisite? -----> dependOn
+-- use of service/capability? -----> consumeService
+-- abstract -> concrete? -----> realize
+-- explicit responsibility? -----> assignResponsibility
|
+-- reusable restriction? -----> constrain
+-- semantic kind? -----> classify
+-- conditions/input -> result? -----> decisionRule
|
+-- top-K/recent-N/best? -----> local selection
+-- pipeline/order? -----> OPEN local structure review
+-- invoke/start? -----> initiate CANDIDATE
|
'-- none fit honestly? -----> diagnosis / pressure / STOP

```

Question pack != automatic classifier

Le domande restringono le interpretazioni ma non sostituiscono la lettura della source. Più risposte "YES" possono indicare proposition distinte che coesistono; non è necessario comprimere tutto in un solo operatore. Al contrario, una domanda senza evidence governata non deve essere forzata a YES o NO.

10 RoleKey: significato e disciplina

CURRENT / PRESERVED

I roleKey definiscono la funzione svolta da un BAREferent *all'interno di una specifica proposition*. Non classificano universalmente il referent.

roleKey	Significato operativo
behavior	identita' riusabile dello specifico transfer quando il comportamento di conveyance deve essere qualificato o target di altre proposition.
source	origine del content trasferito.
destination	destinatario del content trasferito.
content	significato oggetto della conveyance.
actor	soggetto/capability che compie l'azione quando la firma lo richiede o lo ammette.
result	risultato reso disponibile o risultato governato dell'operazione.
input	significato effettivamente usato dall'operazione; non ogni dato disponibile e' automaticamente input.
created	nuovo item/event project-semantic stabilito da create.
observed	significato esistente letto o ispezionato da observe.
subject	identita' la cui state/lifecycle identity e' interessata da transition, oppure subject locale di satisfies.
fromState	stato precedente governato, se espresso dalla fonte.
toState	stato risultante governato richiesto da transition.
correlatedItem	elemento che deve mantenere same-context binding.
correlationContext	identita' che definisce il contesto condiviso della correlation.
referencer	elemento che mantiene un riferimento direzionale verso altro significato.
referenced	target del riferimento.
dependent	significato che richiede uno o piu' prerequisiti.
prerequisite	significato necessario al dependent.
consumer	utilizzatore effettivo di un service/capability.
service	service/capability effettivamente consumato.
provider	provider del service, soltanto se governato; non e' implicito nel servizio.
abstract	significato astratto indipendentemente governato da materializzare.
realization	significato concreto che materializza l'abstract.
responsibleParty	parte cui viene assegnata responsibility/authority.
responsibilityScope	ambito/oggetto della responsibility assegnata.
responsibilityKind	tipo governato di responsibility/authority.
constraintTarget	significato cui si applica una restriction governata.
constraintValue	valore, dominio o struttura di restriction governata.
classifiedReferent	elemento cui viene assegnato un semantic kind.
semanticKind	categoria methodology-neutral governata.

11 Matrice normativa delle cardinalita'

CURRENT / PRESERVED

Operator	Required	Optional / local
transfer	source 1; destination 1..*; content 1..*	behavior 0..1
produce	actor 1; result 1..*	input 0..*

Operator	Required	Optional / local
create	actor 1; created 1..*	none
observe	actor 1; observed 1..*	result 0..*
transition	subject 1; toState 1	actor 0..1; fromState 0..1
correlate	correlatedItem 1..*; correlationContext 1	none
reference	referencer 1; referenced 1..*	none
dependOn	dependent 1; prerequisite 1..*	none
consumeService	consumer 1..*; service 1	provider 0..1
realize	abstract 1; realization 1..*	none
assignResponsibility	responsibleParty 1..*; responsibilityScope 1; responsibilityKind 1	none
constrain	constraintTarget 1; constraintValue 1..*	none
classify	classifiedReferent 1; semanticKind 1..*	none
decisionRule	actor 1; input 1..*; result 1	operator-local rule

12 Operator: transfer

CURRENT / PRESERVED

R25 EVIDENCE

12.1 Definizione

transfer rappresenta la conveyance governata di uno o piu' content da una source a una o piu' destination. Non implica di per se' protocollo, successo, ownership, storage, processing o service consumption.

12.2 Domanda diagnostica

La source governa che un contenuto venga reso disponibile/spostato/consegnato da una origine identificabile a una destinazione identificabile?

12.3 Firma canonica

```
transfer
  behavior    -> BAREferent [0..1]
  source      -> BAREferent [1]
  destination -> BAREferent [1..*]
  content     -> BAREferent [1..*]
```

12.4 Campi

Role	Card.	Semantica
behavior	0..1	identita' del comportamento di transfer, solo se deve essere riusato/qualificato.
source	1	origine governata della conveyance.
destination	1..*	uno o piu' destinatari governati.
content	1..*	contenuto il cui trasferimento e' governato.

12.5 Invarianti e confini

- `transfer != produce`: produrre un risultato non prova che esso venga trasferito a un destinatario.
- `transfer != observe`: leggere qualcosa non prova una conveyance source-to-destination.
- `transfer != consumeService`: utilizzare un service non coincide con trasferire content.
- `transfer != initiate`: avviare/invocare qualcosa non implica conveyance.
- `interaction mode`, `channel`, `retries`, `timeout`, `queueing`, `acknowledgement` e `failure semantics` si aggiungono solo se governati e rappresentabili nel livello corretto.

12.6 Forbidden inference

Non inferire protocollo, sincronicit , cifratura, affidabilit , successo, persistenza, ownership della source/destination o topologia intermedia.

12.7 Worked examples

Facial Access - FR-3.4.2: delivery della RecognitionCapture

Source meaning. CameraSubsystem deve consegnare la RecognitionCapture a RecognitionProcessor preservando il binding con la stessa IdentityDeterminationRequest. La conveyance e il same-request binding sono due significati distinti.

Extraction.

```

BAREferent
  CameraSubsystem
  RecognitionProcessor
  RecognitionCapture
  RecognitionCaptureDelivery
  IdentityDeterminationRequest

BAProposition P1
transfer
  behavior -> RecognitionCaptureDelivery
  source   -> CameraSubsystem
  destination -> RecognitionProcessor
  content   -> RecognitionCapture

BAProposition P2
correlate

```

```

correlatedItem      -> RecognitionCapture
correlationContext  -> IdentityDeterminationRequest

```

Boundary lesson. Il binding non viene schiacciato dentro transfer; resta una proposition correlate separata.

DermaTriage - FR-23: scambio B4 con case identity preservata

Source meaning. Nel workflow B4-integrated, DermaTriage effettua write-back/retrieval mantenendo la stessa consultation/case identity.

```

transfer
  source      -> DermaTriage
  destination -> B4
  content     -> B4TriageOutcome

transfer
  source      -> B4
  destination -> DermaTriage
  content     -> ValidatedOutcome

correlate
  correlatedItem      -> ValidatedOutcome
  correlationContext  -> OriginalDermaTriageOutcome

```

Non inferire: protocollo ulteriore, queue, retry, success outcome o storage semantics se non governati.

13 Operator: produce

CURRENT / PRESERVED

R25 EVIDENCE

13.1 Definizione

produce rappresenta un actor/capability che rende disponibile un risultato governato. Il risultato può già avere identità semantica senza essere “creato” nel senso di nuova occurrence/item project-semantic.

13.2 Domanda diagnostica

La fonte governa che un actor/capability renda disponibile un output/result, indipendentemente dal fatto che tale output costituisca una nuova identità di progetto?

13.3 Firma canonica

```

produce
  actor   -> BAREferent [1]
  result  -> BAREferent [1..*]
  input   -> BAREferent [0..*]

```

13.4 Campi

actor	1	soggetto/capability che produce il risultato.
result	1..*	output/result reso disponibile.
input	0..*	input governati effettivamente usati, non semplicemente disponibili.

13.5 Invarianti e confini

- `produce != create`: output availability non prova nuova identity/occurrence.
- `produce != transfer`: la disponibilit  di un result non identifica source/destination di conveyance.
- `produce != decisionRule`: un risultato prodotto non implica un mapping condizionale governato.
- `produce != observe`: un result di una query puo' essere prodotto, ma la lettura dell'esistente e il result prodotto sono semantiche separabili.

13.6 Forbidden inference

Non inferire algoritmo, score, ranking, persistenza, destinazione, nuova identity o condition logic non esplicitamente governati.

13.7 Worked examples

Facial Access - FR-3.2.1: governed identity-determination outcome

La capability di riconoscimento deve produrre un esito governato distinguendo riuscita, negativa e non conclusiva.

```
produce
  actor  -> RecognitionProcessor
  input  -> RecognitionCapture
  result -> IdentityDeterminationOutcome
```

Perche' produce. La clausola governa la disponibilit  del risultato. Non stabilisce che l'outcome sia una nuova identity project-semantic creata da create.

DermaTriage - FR-16 e FR-19: due produzioni con input diversi

```
produce
  actor  -> EfficientNet-B4
  input  -> SkinLesionImage
  result -> ImageUrgencyClassification

produce
  actor  -> BioMistral-7B
  input  -> ImageUrgencyClassification
  input  -> ClinicalDescription
  input  -> HistoricalCaseContext
```

```
input  -> AvailableSymptomInformation
result -> AITriageSynthesis
```

Il secondo caso mostra che input [0..*] identifica solo gli input effettivamente governati, non ogni informazione disponibile nel sistema.

14 Operator: create

CURRENT / PRESERVED

R25 EVIDENCE

14.1 Definizione

create rappresenta l'istituzione di una nuova identità/occurrence/item project-semantic che prima dell'azione non esisteva nel significato governato.

14.2 Domanda diagnostica

La source governa esplicitamente che l'azione stabilisca una nuova occurrence o identità' rilevante per il progetto?

14.3 Firma canonica

```
create
  actor    -> BAReferent [1]
  created  -> BAReferent [1..*]
```

14.4 Campi

actor	1	soggetto/capability che stabilisce la nuova identity/occurrence.
created	1..*	item/event project-semantic creato.

14.5 Invarianti e confini

- create != produce: generare un output non dimostra nuova identity.
- create != transition: cambiare stato dello stesso subject non crea un nuovo subject.
- create != storedIn: persistenza/at-rest association non prova creazione.
- nomi tecnici come factory, generator, constructor o insert non sono sufficienti.

14.6 Forbidden inference

Non inferire ID generation, storage technology, lifecycle successivo, ownership o correlation con altri elementi se non governati.

14.7 Worked examples e negative control

Nessun positive control pulito nei due corpus principali

Facial Access e DermaTriage esercitano molto bene produzione, transfer, correlation, selection e decision semantics, ma non offrono nel current source set un caso incontestabile in cui una nuova identity project-semantic debba essere modellata con create. La guida non forza quindi un esempio artificiale dal corpus.

DIDATTICO NON NORMATIVO - creazione di un account

Source:
dopo approvazione, AccountProvisioning crea un nuovo UserAccount

```
create
  actor    -> AccountProvisioning
  created  -> UserAccount
```

Delete test. Se si sostituisse create con produce, si perderebbe il fatto materiale che dopo l'azione esiste una nuova identity/occurrence di progetto.

DermaTriage - perche' FR-20 NON prova create

FR-20 governa un training process che produce/seleziona un best checkpoint. Il fatto che emerga un checkpoint non autorizza automaticamente a modellarlo come nuova identity project-semantic con create; prima deve essere governata proprio quella distinzione di identity/occurrence.

15 Operator: observe

CURRENT / PRESERVED

R25 EVIDENCE

15.1 Definizione

observe rappresenta read/query/inspection esplicita di significato esistente senza asserire che l'osservazione crei o modifichi l'oggetto osservato.

15.2 Domanda diagnostica

La fonte governa un atto di lettura, query o ispezione di qualcosa che esiste gia' come significato rilevante?

15.3 Firma canonica

```
observe
  actor    -> BReferent [1]
  observed -> BReferent [1..*]
  result   -> BReferent [0..*]
```


15.4 Campi

actor	1	soggetto/capability che legge o ispeziona.
observed	1..*	significato esistente oggetto dell'osservazione.
result	0..*	eventuale result distinto prodotto dall'osservazione; CMD-OP04 ne mantiene aperta la revisione.

15.5 Invarianti e confini

- `observe != input`: il fatto che un dato sia input non prova un'operazione di lettura semanticamente governata.
- `observe != transfer`: ricevere content non equivale automaticamente a osservarlo.
- `observe != selection`: query/retrieval puo' avvenire dopo una selection; non stabilisce la membership selezionata.
- `observe != transition`: inspection non implica state change.

15.6 Open delta

OPEN / COUNTEREXAMPLE-GATED CMD-OP04: la rimozione di `observe.result` resta deferita a integrated review; questa candidate preserva quindi la firma R3.

15.7 Worked examples e confine con selection

DermaTriage - FR-18: query/retrieval dopo la selection

FR-18 governa embedding, cosine similarity, top-5 e recupero dei casi storici. La selection stabilisce quali membri appartengono al selected set; l'eventuale query/read degli item gia' selezionati e' una semantica distinta.

```
selection                                # local candidate structure
  candidatePopulation -> HistoricalCases
  selectionBasis      -> CosineSimilarity
  selectedResult      -> HistoricalCaseContext
  bound               -> 5
  orderingBasis       -> CosineSimilarityDescending

observe                                # only for the read/query meaning
  actor               -> DermaTriageRetrievalCapability
  observed            -> HistoricalCaseContext
```

Boundary. observe non sostituisce la selected-membership semantics.

DIDATTICO NON NORMATIVO - lettura di stato esistente

```
observe
  actor      -> MonitoringAgent
  observed   -> CurrentDoorState
```

Non inferire transition o creazione dello stato osservato.

16 Operator: transition

CURRENT / PRESERVED

R25 EVIDENCE

16.1 Definizione

transition rappresenta un cambiamento governato di stato/lifecycle dello *stesso* subject project-semantic.

16.2 Domanda diagnostica

La fonte governa che la stessa identita' passi a uno stato/lifecycle differente?

16.3 Firma canonica

```
transition
  subject    -> BAREferent [1]
  toState    -> BAREferent [1]
  actor      -> BAREferent [0..1]
  fromState  -> BAREferent [0..1]
```

16.4 Campi

subject	1	identita' che persiste attraverso il cambiamento.
toState	1	stato risultante governato.
actor	0..1	soggetto che causa/realizza il cambiamento, solo se governato.
fromState	0..1	stato precedente governato, se noto.

16.5 Invarianti e confini

- stessa identity prima/dopo e' requisito centrale;
- selection di una versione non e' automaticamente transition;
- qualification result non e' automaticamente state transition;
- modifica di una property non prova lifecycle/state identity;
- supporto al rollback non prova che una transition specifica sia governata o automatica.

16.6 Open delta

OPEN / COUNTEREXAMPLE-GATED CMD-OP05: la distinzione state identity vs typed value resta deferita; nessuna estensione della firma R3 viene ammessa qui.

16.7 Worked examples e boundary control

DermaTriage - FR-10: rollback support non basta da solo

FR-10 richiede di supportare la revoca di un adattamento degradante mediante il ripristino di una versione o stato precedente accettabile. Questa source pressure e' utile ma non autorizza automaticamente una proposition transition: resta necessario stabilire quale *stessa identity* attraversi quali stati governati.

```
NOT AUTOMATICALLY VALID
transition
  subject    -> CandidateModelVersion    # identity not yet established here
  fromState  -> Adopted
  toState    -> Revoked
```

La guida mantiene quindi il boundary: rollback support, version selection e state transition non sono sinonimi.

DIDATTICO NON NORMATIVO - transition pulita

```
transition
  actor      -> AccountAdministration
  subject    -> AccessCredential
  fromState  -> ActiveCredentialState
  toState    -> RevokedCredentialState
```

Qui la stessa `AccessCredential` persiste attraverso il cambiamento.

17 Operator: correlate

CURRENT / PRESERVED

R25 EVIDENCE

17.1 Definizione

`correlate` preserva un binding di same-context tra uno o piu' elementi e una identita' di contesto condivisa.

17.2 Domanda diagnostica

La source richiede che piu' significati appartengano alla stessa request, identity, case, evaluation, attempt, operation o altro contesto governato?

17.3 Firma canonica

```
correlate
  correlatedItem      -> BReferent [1..*]
  correlationContext -> BReferent [1]
```

17.4 Campi

correlatedItem	1..*	elemento/i che devono restare nello stesso contesto.
correlationContext	1	identita' che definisce il contesto condiviso.

17.5 Invarianti e confini

- `correlate != reference`: il reference puo' essere direzionale senza same-context invariant.
- `correlate != dependOn`: same-context binding non implica prerequisite.
- `correlate != transfer`: binding non implica conveyance.
- non usare correlate come generic association fallback.

17.6 Worked examples

Facial Access - stessa request durante il delivery

```
correlate
  correlatedItem      -> RecognitionCapture
  correlationContext -> IdentityDeterminationRequest
```

La parola semantica decisiva e' *stessa*: il requisito richiede same-request binding, non un generico riferimento.

DermaTriage - FR-03 e FR-23: stessa review/case identity

```
correlate
  correlatedItem      -> ClinicalReviewResult
  correlationContext -> OriginalDermaTriageOutcome

correlate
  correlatedItem      -> ValidatedOutcome
  correlationContext -> OriginalDermaTriageOutcome
```

Confine. Se basta sapere che A "riguarda" B si valuta reference; quando la stessa case identity deve essere preservata si usa correlate.

18 Operator: reference

CURRENT / PRESERVED

R25 EVIDENCE

18.1 Definizione

reference rappresenta un riferimento direzionale esplicito da un referencer a uno o piu' target identificati, quando non e' governata una semantica piu' forte.

18.2 Domanda diagnostica

La fonte governa che A riguardi/riciami/punti a B, senza richiedere same-context binding, prerequisite, ownership, creation o transfer?

18.3 Firma canonica

```
reference
  referencer -> BReferent [1]
  referenced -> BReferent [1..*]
```

18.4 Campi

referencer	1	elemento che mantiene il riferimento.
referenced	1..*	target del riferimento.

18.5 Invarianti e confini

- `reference != correlate`: non garantisce same-context binding.
- `reference != dependOn`: non garantisce prerequisite.
- `reference != assignResponsibility`: non assegna authority/responsibility.
- non usare `reference` come fallback per relazioni semanticamente non comprese.

18.6 Worked examples

Facial Access - authorization state concerning GovernedIdentity

```
reference
  referencer -> AccessAuthorizationState
  referenced -> GovernedIdentity
```

La proposition dice che lo stato riguarda quella identity; non implica same-request binding, ownership o un boolean `authorized = TRUE`.

DermaTriage - FR-09: candidate valutato rispetto a una reference applicabile

```
reference
  referencer -> CandidateUrgencyClassificationAdaptation
  referenced -> ApplicableReferenceVersion
```

Questa relation puo' coesistere con una `decisionRule` che usa i criteri comparativi. La `reference` identifica il target del confronto; non rappresenta da sola il mapping di qualificazione.

19 Operator: dependOn

CURRENT / PRESERVED

R25 EVIDENCE

19.1 Definizione

dependOn rappresenta una relazione di prerequisite: il dependent richiede il prerequisite per il significato governato in scope.

19.2 Domanda diagnostica

La fonte governa che A non possa soddisfare/completare il proprio significato senza B, oppure che B sia una preconditione esplicita per A?

19.3 Firma canonica

```
dependOn
  dependent    -> BAREferent [1]
  prerequisite -> BAREferent [1..*]
```

19.4 Campi

dependent	1	significato che necessita del prerequisite.
prerequisite	1..*	prerequisite governati richiesti.

19.5 Invarianti e confini

- temporal order alone non e' dependency;
- precedes != dependOn;
- pipeline membership non e' dependency;
- dependOn != consumeService: un prerequisite puo' non essere un service consumato;
- la transitive closure puo' essere projection-derived e non deve essere materializzata come truth senza necessita'.

19.6 Worked examples

Facial Access - delivery come prerequisite della determinazione

D-3.4 governa che l'indisponibilita' dell'interazione possa impedire il completamento della determinazione dell'identita'.

```
dependOn
  dependent    -> IdentityDeterminationOutcome
  prerequisite -> RecognitionCaptureDelivery
```

DermaTriage - MR-04 dependsOn MR-03

```

dependOn
  dependent    -> ControlledAdaptation
  prerequisite -> ClinicalReviewMeaning

```

La dependency esprime che l'adaptation branch consuma significato governato dal review branch. Non crea parentage o co-ownership dei Requirement.

Ordine non sufficiente

Che uno stadio B venga dopo A nella pipeline non basta per creare `dependOn(A,B)`. Serve prerequisite semantics governata.

20 Operator: consumeService

CURRENT / PRESERVED

R25 EVIDENCE

20.1 Definizione

`consumeService` rappresenta l'uso effettivo di un service/capability da parte di uno o piu' consumer, senza trasferire automaticamente ownership o provider responsibility.

20.2 Domanda diagnostica

La fonte governa che un consumer utilizzi effettivamente una capability/service fornita come tale?

20.3 Firma canonica

```

consumeService
  consumer -> BAREferent [1..*]
  service  -> BAREferent [1]
  provider -> BAREferent [0..1]

```

20.4 Campi

consumer	1..*	utilizzatore/i effettivi del service.
service	1	service/capability consumato.
provider	0..1	provider esplicitamente governato; non implicito.

20.5 Invarianti e confini

- endpoint, componente, tecnologia, dataset o information item non sono automaticamente service;
- `consumeService != transfer;`
- `consumeService != provideService candidate;`

- consumption non implica ownership, administration o infrastructure responsibility;
- un consumer requirement non prova che il service garantisca la property richiesta.

20.6 Worked examples

Facial Access - consumo del servizio di connettività

```
consumeService
  consumer -> IdentityDeterminationRealization
  service  -> ConnectivityService
```

Il provider resta assente se non e' governato. Il consumo non trasferisce ownership dell'infrastruttura al consumer.

DermaTriage - B4 come sistema/servizio esterno consumato

Nel workflow integrato, DermaTriage acquisisce consultation, documenti e validated outcomes da B4, che resta sistema esterno. Quando la source governa effettivamente l'uso della capability B4, la lettura puo' essere:

```
consumeService
  consumer -> DermaTriageB4Integration
  service  -> B4ExternalService
```

Le singole informazioni scambiate restano invece content di transfer o input di altre proposition; non diventano service.

21 Operator: realize

CURRENT / PRESERVED

R25 EVIDENCE

21.1 Definizione

realize rappresenta la materializzazione concreta di un significato astratto che esiste indipendentemente come commitment/capability/contract governato.

21.2 Domanda diagnostica

Esiste un significato astratto governato indipendentemente e la source governa che uno o piu' elementi concreti lo materializzino?

21.3 Firma canonica

```
realize
  abstract      -> BAREferent [1]
  realization   -> BAREferent [1..*]
```


21.4 Campi

abstract	1	significato astratto indipendentemente governato.
realization	1..*	elemento/i concreti che lo materializzano.

21.5 Invarianti e confini

- `realize != perform/execute`: realizzare una capability non equivale a rappresentare l'atto di eseguirla;
- `realize != classify`: non assegna semplicemente un kind;
- `realize != assignResponsibility`: materialization non equivale a responsibility placement;
- deve esistere un abstract meaning indipendentemente governato.

21.6 Worked examples

Facial Access - facial recognition realizes identity determination

```
realize
  abstract    -> IdentityDetermination
  realization -> FacialRecognitionStrategy
```

La responsabilit  astratta resta valida anche se in futuro la strategy concreta cambia.

DermaTriage - concrete pipeline vs abstract triage responsibility

MR-01 governa la responsabilit  di valutazione di triage; DEC-12 governa una pipeline AI image-based concreta. Quando la relazione abstract-to-concrete   materialmente necessaria alla BA,   possibile preservarla come:

```
realize
  abstract    -> DermatologicalTriageEvaluation
  realization -> ImageBasedFourStagePipeline
```

Non usare `realize` per rappresentare semplicemente che EfficientNet-B4 "esegue" una classificazione: execution semantics non sono assorbite da questo operatore.

22 Operator: assignResponsibility

CURRENT / PRESERVED

R25 EVIDENCE

22.1 Definizione

`assignResponsibility` colloca una responsibility o authority governata su una o pi  responsible party per uno scope e un responsibility kind espliciti.

22.2 Domanda diagnostica

La fonte assegna esplicitamente responsabilit /authority a una parte, distinguendo tale assegnazione dal semplice fatto che la parte produca, fornisca, consumi o esegua qualcosa?

22.3 Firma canonica

```
assignResponsibility
  responsibleParty    -> BAREferent [1..*]
  responsibilityScope -> BAREferent [1]
  responsibilityKind  -> BAREferent [1]
```

22.4 Campi

responsibleParty	1..*	parte/i cui � attribuita la responsibility.
responsibilityScope	1	ambito governato della responsibility.
responsibilityKind	1	tipo di responsibility/authority governato.

22.5 Invarianti e confini

- provider != responsible party per default;
- performer/actor != responsible party per default;
- producer/consumer != responsible party per default;
- realization != responsibility;
- mera partecipazione non assegna authority.

22.6 Worked examples e responsibility boundary

Facial Access - D-3.4: acquisition e recognition hanno owner distinti

La Decision separa CameraSubsystem e RecognitionProcessor come responsibility owner distinti. Quando il tipo di responsabilit  e lo scope sono governati in modo sufficiente, la BA puo' esplicitare:

```
assignResponsibility
  responsibleParty    -> CameraSubsystem
  responsibilityScope -> RecognitionCaptureAcquisition
  responsibilityKind  -> OperationalResponsibility

assignResponsibility
  responsibleParty    -> RecognitionProcessor
  responsibilityScope -> FacialRecognitionProcessing
  responsibilityKind  -> OperationalResponsibility
```

DermaTriage - B4 non acquisisce la responsibility del triage

DEC-14 stabilisce che B4 partecipa allo scambio dati ma non acquisisce la responsibility del triage posseduta da DermaTriage. Questo e' soprattutto un *negative responsibility boundary*: non deve essere invertito inventando una responsibility positiva per B4.

Actor != responsible party

Che un component produca un risultato o invochi un endpoint non basta a dichiararlo responsible party. `assignResponsibility` richiede responsibility/authority semantics esplicita.

23 Operator: constrain

CURRENT / PRESERVED

R25 EVIDENCE

23.1 Definizione

`constrain` rappresenta una restriction governata, riusabile e interrogabile applicata a un target identificato.

23.2 Domanda diagnostica

La fonte governa una restriction stabile su un target, distinta da una semplice condition locale, da un mapping decisionale o da un bound interno a una selection?

23.3 Firma canonica

```
constrain
  constraintTarget -> BAREferent [1]
  constraintValue  -> controlled value/structure [1..*]
```

23.4 Campi

<code>constraintTarget</code>	1	significato esatto su cui insiste la restriction.
<code>constraintValue</code>	1..*	valore, dominio o struttura di constraint governata.

23.5 Invarianti e confini

- `constrain != decisionCondition`: una condition locale non e' automaticamente reusable constraint;
- `constrain != decisionRule`: un dominio ammesso non definisce necessariamente come scegliere un risultato;
- selection bound puo' coesistere con `constrain` ma non assorbe selected membership;
- `NOT_GOVERNED` non viene trasformato in constraint negativo.

23.6 Worked examples

Facial Access - D-3.6: wired Ethernet sul delivery specifico

```

constrain
  constraintTarget -> RecognitionCaptureDelivery
  constraintValue
    property -> interconnectionMedium
    vocabulary -> [WIRED_ETHERNET]

```

Il constraint e' scoped al delivery governato; non si propaga all'intero provider infrastructure.

DermaTriage - FR-24: meccanismo richiesto per operazioni amministrative protette

```

constrain
  constraintTarget -> ProtectedDermaTriageOperation
  constraintValue
    property -> authenticationMechanism
    value -> X-API-Key

```

La stessa sintassi concettuale puo' descrivere un dominio o un valore governato. Il constraint non equivale all'atto di autenticare una request e non crea da solo un service relation.

24 Operator: classify

CURRENT / PRESERVED

R25 EVIDENCE

24.1 Definizione

classify assegna a un referent uno o piu' semantic kind/category methodology-neutral governati.

24.2 Domanda diagnostica

La fonte governa che un elemento appartenga a una categoria/kind semanticamente rilevante, indipendentemente da una selection tra candidati o da un lifecycle change?

24.3 Firma canonica

```

classify
  classifiedReferent -> BReferent [1]
  semanticKind -> BReferent [1..*]

```

24.4 Campi

classifiedReferent	1	elemento cui viene assegnato il kind.
semanticKind	1..*	categoria/kind governati.

24.5 Invarianti e confini

- `classify != selection`: assegnare un kind non sceglie un membro tra candidati;
- `classify != transition`: cambiare category non prova lifecycle state change;
- `classify != generic property assignment`;
- il kind deve essere methodology-neutral e materialmente utile, non introdotto solo per il tool.

24.6 Worked examples

DermaTriage - FR-12: confirmation vs correction

```
classify
  classifiedReferent -> ClinicalReviewResult
  semanticKind      -> ConfirmationOrCorrection
```

L'operatore conserva il kind/disposition semantico. Non rappresenta una selection tra candidate review e non implica lifecycle transition.

DermaTriage - FR-16: urgency class e produce devono restare distinti

FR-16 governa sia la produzione di `ImageUrgencyClassification` sia il dominio HIGH/MEDIUM/LOW. Una BA puo' aver bisogno di:

```
produce
  actor  -> EfficientNet-B4
  input  -> SkinLesionImage
  result -> ImageUrgencyClassification

constrain / classify
  # scelta dipende da cosa e' governato come kind vs allowed domain
```

La guida non trasforma automaticamente ogni output label in `classify`: prima va stabilito se la semantica e' category identity o semplice value domain.

25 Operator: decisionRule

CURRENT / PRESERVED

R25 EVIDENCE

25.1 Definizione

`decisionRule` rappresenta un mapping governato da input/conditions a un risultato governato. E' appropriato quando il progetto governa non soltanto il dominio dei risultati, ma la regola che determina il risultato.

25.2 Domanda diagnostica

La fonte governa una regola esplicita che, dati input/conditions, determina uno specifico result o result assignment?

25.3 Firma canonica

```
decisionRule
  actor  -> BAREferent [1]
  input  -> BAREferent [1..*]
  result -> BAREferent [1]

  rule
    IF    decisionCondition
    THEN  resultAssignment [1..*]
    ELSE  resultAssignment [0..*]
```

25.4 Campi top-level

actor	1	soggetto/capability che applica la regola.
input	1..*	input governati effettivamente usati dal mapping.
result	1	risultato governato dal rule mapping.
rule	1 local structure	struttura operator-local di condition e assignment.

25.5 Invarianti e confini

- conditional action non e' automaticamente decisionRule;
- non-sufficiency statement non e' automaticamente decisionRule;
- ranking/selection non e' automaticamente decisionRule;
- capability/support statement non e' automaticamente decisionRule;
- un allowed result domain da solo non definisce il mapping;
- property/value interni non vengono inventati per soddisfare la sintassi.

25.6 Worked examples

Facial Access - AccessDecision condizionata a identity outcome e authorization condition

```
decisionRule
  actor  -> ControlledAreaAccess
  input  -> IdentityDeterminationOutcome
  input  -> AccessAuthorizationState
  result -> AccessDecision

  rule
    IF allof(
      comparison(outcomeKind equals SUCCESS),
      satisfies(AccessAuthorizationState,
        RequiredAccessAuthorizationCondition))
    THEN AccessDecision = ALLOWING_ACCESS_DECISION
    ELSE AccessDecision = NON_ALLOWING_ACCESS_DECISION
```

L'uso di satisfies evita di inventare un boolean interno authorized = TRUE non governato.

DermaTriage - FR-02: mapping P-scale

La documentazione governa un mapping esplicito:

```
HIGH + confidence > 0.85 -> P1
HIGH                               -> P2
MEDIUM                           -> P3
LOW                               -> P4
```

Una representation candidate usa decisionRule per preservare il mapping, con ordered comparison demand su confidence > 0.85. Il fatto che la pressione > sia reale non autorizza a congelare qui un nuovo comparisonKey: CL-01 resta OPEN sul controlled vocabulary ordinato.

DermaTriage - FR-09: tutti i criteri applicabili devono essere soddisfatti

```
decisionRule
  actor  -> DermaTriage
  input  -> CandidateUrgencyClassificationAdaptation
  input  -> ApplicableReferenceVersion
  result -> CandidateQualificationResult

  rule
    IF allof(applicableAcceptanceCriteria)
    THEN CandidateQualificationResult = QUALIFIED
```

Boundary. La qualification non implica deployment automatico.

26 Condition language di decisionRule

CURRENT / PRESERVED

R25 EVIDENCE

Le condition form sono strutture locali. Non aumentano il conteggio dei top-level operator.

26.1 comparison

```
comparison
  referent      -> BReferent [1]
  property      -> controlled semantic key [1]
  comparisonKey -> equals | notEquals
  value         -> typed local value | BReferent [1]
```

referent	elemento la cui property viene confrontata.
property	semantic key controllata; resta obbligatoria nel contratto corrente.
comparisonKey	operazione di confronto ammessa. Current: equals, notEquals.
value	valore tipizzato o BReferent contro cui avviene il confronto.

CL-01 - ordered/scalar comparison vocabulary

Il ciclo R25 ha prodotto forte evidence per confronti ordinati/scalari che superano equals/notEquals. Le relazioni concettuali <, >, <=, >= sono pressure reale, ma il control-

led vocabulary normativo e la relativa typing discipline non sono ancora chiusi. Non usarli come vocabulary BA2 current senza formal delta.

26.2 satisfies

CURRENT / PRESERVED

```
satisfies
  subject    -> BAREferent [1]
  condition  -> BAREferent [1]
```

Serve quando la fonte governa che un subject soddisfi una condition identificata, ma non governa un modello property/value interno che autorizzi una comparison.

26.3 allof

CURRENT / PRESERVED

R25 EVIDENCE

```
allof
  childCondition -> decisionCondition [2..*]
```

Tutte le condizioni figlie devono valere. R25 fornisce forte evidence di conjunction semantics, senza trasformare allof in top-level operator.

26.4 anyOf

CURRENT / PRESERVED

```
anyOf
  childCondition -> decisionCondition [2..*]
```

Almeno una condizione figlia deve valere. Il costrutto e' gia' corrente, ma R25 lo mantiene in stato PENDING: il positive control e' ancora insufficiente. Questo non equivale a rimozione dal contratto corrente.

26.5 not

CURRENT / PRESERVED

```
not
  childCondition -> decisionCondition [1]
```

Negazione locale di una condition. La frase "A non e' sufficiente per B" non equivale a not (B) e non deve essere compressa in una negazione logica falsa.

26.6 resultAssignment

CURRENT / PRESERVED

```
resultAssignment
```



```
target -> BAREferent [1]
value  -> typed local value | BAREferent [1]
```

Rappresenta l'assegnazione del risultato governato nel ramo THEN/ELSE. Non autorizza property assignment generico fuori da decisionRule.

26.7 Worked examples della condition language

comparison - DermaTriage FR-02 e DEC-07

```
comparison
  referent      -> ImageUrgencyClassification
  property      -> confidence
  comparisonKey -> GREATER_THAN      # evidence need; key not yet frozen
  value         -> 0.85
```

L'evidence dimostra la necessita' di ordered comparison, non il nome definitivo del controlled key. Per equality/inequality restano current equals e notEquals.

satisfies - Facial Access

```
satisfies
  subject      -> AccessAuthorizationState
  condition    -> RequiredAccessAuthorizationCondition
```

La forma preserva "soddisfa la condizione" senza inventare una property interna.

allOf - DermaTriage FR-09

FR-09 richiede qualification soltanto se *tutti* i criteri comparativi governati applicabili risultano soddisfatti. Questo e' un positive control forte per allOf.

anyOf - R25 insufficiently tested

anyOf resta parte della condition language current, ma il secondo ciclo non ha fornito un controllo altrettanto forte. La guida preserva la sintassi senza attribuirle una nuova claim R25.

not - negative boundary da DermaTriage

La frase "un risultato del path A non e' sufficiente, da solo, a determinare il risultato del path B" **non** equivale a not(resultB). Non-sufficiency e negazione logica sono semantiche diverse.

27 Struttura locale candidate: selection

CANDIDATE / NOT ADMITTED

R25 EVIDENCE

R25 ha dimostrato che bounded/ranked/recency selection contiene semantica non eliminabile in diversi controlli, senza giustificare un nuovo top-level operator.

27.1 Working structure

```
selection
  candidatePopulation -> OPEN TYPE [required]
  selectionBasis      -> OPEN TYPE [required]
  selectedResult      -> OPEN TYPE [required]
  bound               -> OPEN TYPE [optional, only if governed]
  orderingBasis       -> OPEN TYPE [optional, only if governed]
```

Typing non ancora congelato

Questa struttura e' REUSABLE_SELECTION_STRUCTURE / NON-NORMATIVE. R25 non decide ancora se i campi siano BAREferent, local structured values, condition-language structures o altro tipo gia' governato. criterion e selectionBasis restano una naming decision da chiudere nell'integrazione metodologica.

27.2 Distinzioni obbligatorie

```
ranking      != selection
bound        != selection
retrieval    != selection
filtering    != selection
best selection != full ranking
```

La membership selezionata deve restare distinguibile dal criterio, dall'eventuale ordinamento e dall'eventuale bound.

27.3 Worked comparison: tre forme diverse di selection in DermaTriage

FR-18 - ranked top-5 historical cases

```
selection
  candidatePopulation -> HistoricalCases
  selectionBasis      -> CosineSimilarity
  selectedResult      -> HistoricalCaseContext
  bound               -> 5
  orderingBasis       -> CosineSimilarityDescending
```

Qui ranking, bound e selected membership sono tutti materialmente rilevanti.

FR-06 - recent-N pertinent corrections

```
selection
  candidatePopulation -> ClinicalCorrections
  selectionBasis      -> PertinenceAndRecency
  selectedResult      -> PromptEvolutionEvidenceSet
  bound               -> 20
  orderingBasis       -> Recency # only to the extent actually governed
```

Deduplication, underfill, reuse e overlap restano source gaps e non vengono inventati.

FR-20 - best checkpoint by validation Macro F1

```

selection
  candidatePopulation -> TrainingCheckpoints
  selectionBasis      -> ValidationMacroF1
  selectedResult     -> BestBaselineCheckpoint

```

Key lesson: best/argmax-like selection non implica una full ranking riusabile di tutti i candidate.

28 Pressure aperta: pipeline composition e order

OPEN / COUNTEREXAMPLE-GATED

R25 EVIDENCE

PR-02 PIPELINE_COMPOSITION_ORDER resta aperta. La guida deve quindi preservare le distinzioni senza introdurre prematuramente pipeline o precedes come top-level operator.

```

pipeline membership
  != pipeline ordering
  != precedes
  != dependOn
  != transfer
  != initiate

```

28.1 Domande da porre

- La fonte governa soltanto appartenenza a una stessa composizione?
- Governa un ordine totale o parziale?
- L'ordine e' semantico o soltanto descrittivo/implementativo?
- B richiede A come prerequisite, oppure A e' soltanto precedente?
- Esiste conveyance tra stadi oppure solo successione logica?
- La composizione richiede una reusable local structure oppure e' proiettabile da proposition gia' accettate?

Current disposition

Nuovo top-level operator: NOT JUSTIFIED. Reusable local composition structure: possibile, ma non ancora chiusa. Test successivo: membership vs ordering; precedes vs dependOn.

28.2 Worked boundary: DermaTriage DEC-12

DEC-12 governa una pipeline sequenziale a quattro stadi: EfficientNet-B4, Qwen2-VL, historical retrieval e BioMistral-7B. Questa evidence richiede di preservare almeno membership e ordine governato, ma non autorizza a trasformare automaticamente ogni adiacenza in dependOn o transfer.

```

PIPELINE MEMBERSHIP
  EfficientNet-B4
  Qwen2-VL-7B-Instruct

```

HistoricalRetrieval
BioMistral-7B

ORDERING PRESSURE

stage 1 before stage 2 before stage 3 before stage 4

NOT AUTOMATICALLY

dependOn(stage2, stage1)
transfer(stage1, stage2)
initiate(stage1, stage2)

Quando una downstream stage usa esplicitamente un output precedente, quella semantica viene rappresentata separatamente tramite i propri operatori/ruoli. La structure di pipeline rimane OPEN come representation question, non come nuovo top-level operator già ammesso.

29 Pressure e strutture trasversali ancora aperte

ID	Topic	Stato / disciplina
PR-01	function/process/behavior identity / perform?	PRESSURE RETAINED; execution evidence esiste ma irreducibility non dimostrata.
PR-04	boundary/interaction association	OPEN; distinguere crossing, transfer, interface e initiation.
PR-07	structured information contract	OPEN; puo' appartenere a structure/- schema/composition anziche' dynamic relation.
PR-09	acquisition/refresh semantics	OPEN; esaurire operatori correnti prima di introdurre primitive.
PR-12	negative implication/non- sufficiency	OPEN; non comprimere in not.
PR-14	applicability/configuration binding	OPEN; provare qualifier/reusable structure prima di nuovo operatore.
OBS-OT-01	operation target / effect scope	LEVEL UNRESOLVED; verificare local structure/composition.

30 Candidate construct R25 non ammessi

30.1 CC-01 consumeData

REJECTED

Disposition: REJECT_REDUNDANT. Nei casi correnti il non-destructive data/evidence use e' rappresentabile senza una nuova primitive top-level; destructive-consume semantics non risultano governate.

30.2 CC-02 provideService

CANDIDATE / NOT ADMITTED

Positive provider/service evidence sopravvive l'exhaustion corrente, ma formal gate G1-G8, compatibility

e final signature sono pendenti.

Signature non congelata

Questa guida non inventa roleKey o cardinalita' di provideService. Fino alla formal admission, il candidate resta evidence e non vocabulary BA2 corrente.

30.3 CC-03 storedIn

CANDIDATE / NOT ADMITTED

L'associazione at-rest sopravvive l'exhaustion corrente, ma formal admission, compatibility e final signature restano pendenti.

Signature non congelata

Questa guida non inventa un modello container/item, storage owner, persistence semantics o lifecycle. storedIn resta TESTED_POSITIVE / NOT_ADMITTED.

30.4 CC-04 initiate

CANDIDATE / NOT ADMITTED

30.4.1 Working minimum signature

```
initiate
  initiator -> BAReferent [1]
  initiated -> BAReferent [1]
```

30.4.2 Semantica minima

Rappresenta una relazione diretta in cui un referent avvia/richiede l'avvio di un altro significato governato. La signature non distingue ancora request vs start come kind separati.

30.4.3 Non implica

```
success
completion
transfer
service consumption
create
transition
automatic trigger
synchronous execution
```

Disposition: TESTED_POSITIVE / NOT_ADMITTED; formal G1-G8 e cross-corpus compatibility sono ancora richiesti.

30.5 Worked candidate examples R25

CC-04 initiate - DermaTriage FR-21

FR-21 governa che un `DirectClient` usi `POST /analyze` per avviare il processo di triage. La working minimum signature e':

```
initiate
  initiator -> DirectClient
  initiated -> DermaTriageTriageProcess
```

Questa evidence non implica success, completion, transfer o service consumption. **Status: TESTED POSITIVE / NOT ADMITTED.**

CC-02 provideService - provider/service relation

Quando una source governa il fatto che una parte *fornisce* una capability come service, la relation provider-service puo' essere semanticamente indipendente dal fatto che un consumer specifico la usi. R25 ha trovato positive evidence per `provideService`, ma l'operatore non e' ammesso e non deve essere usato nella BA current senza formal gate.

CC-03 storedIn - at-rest association

DermaTriage contiene evidence di version/prompt persistence che ha mantenuto viva una relation at-rest distinta da create/transfer/observe. R25 la conserva come `storedIn` candidate, **TESTED POSITIVE / NOT ADMITTED**; la firma definitiva resta fuori da questa guida current.

31 Working hypothesis respinte

REJECTED

Hypothesis	Disposition corrente
generic consume	NOT JUSTIFIED.
generic modify	REJECTED - false semantics come generic operator.
generic persist	NOT JUSTIFIED AS GENERIC OPERATOR.
generic expose	NOT JUSTIFIED - current corpus decomposable.
generic trigger	NOT JUSTIFIED AS SEPARATE PRIMITIVE; directed activation pressure routed to candidate <code>initiate</code> + condition.
verb-specific invoke	ABSORBED BY CC-04 <code>initiate</code> .
initiationKind	REJECT REDUNDANT / OVER-SPECIFIC nel current
REQUEST START	evidence set.

32 BA3 - provenance, origin state e revalidation

CURRENT / PRESERVED

Ogni BA element meaning-bearing deve risolvere a una baseline governata e alla provenance richiesta dal contratto BA3.

32.1 Origin states

GROUNDED
 DERIVED
 DIAGNOSTIC_UNRESOLVED

32.2 Tre responsabilita' distinte

sourceLink
 dove e' supportato il significato

 derivationBasisBinding
 quali elementi governati e quali ruoli fondano una derivazione

 revalidationContext
 quale change/review context determina freshness e revalidation

Provenance non deve simulare una relazione BA2 mancante.

32.3 Derived semantics

Un elemento DERIVED richiede almeno:

- derivation rule identificabile e versionata;
- basis role-bound;
- applicability contract;
- output semantic contract;
- lineage fino a governed source.

32.4 Change e revalidation

source change
 -> explicit impact / revalidation context
 -> impacted BA elements
 -> PENDING_REVIEW / STALE where applicable
 -> RETAIN / REPLACE / RETIRE
 -> accepted BA for new baseline
 -> rebuild projections

Una accepted BA storica non viene mutata silenziosamente come se le analisi precedenti avessero usato la nuova semantica.

33 Diagnostics e unresolved semantics

CURRENT / PRESERVED

Vocabolario operativo:

MULTIPLE MATERIAL BA CANDIDATES
 BA REQUIRES UNSUPPORTED FACT
 BA LOSES GOVERNED MATERIAL DISTINCTION
 BA EXPOSES SOURCE CONFLICT
 NOT SPECIFIED

NOT SPECIFIED

NOT SPECIFIED non significa false, absent o denied. Significa che il significato necessario non e' determinato dalla baseline governata corrente.

33.1 Quando preferire una diagnosis a una proposition

Usare una diagnosis quando la source non supporta una scelta univoca tra interpretazioni materialmente differenti, quando la firma corrente richiederebbe un fatto non governato o quando il metodo perderebbe una distinzione reale.

34 BA5 - authoring canonico

CURRENT / PRESERVED

I semantic key devono essere canonici nel proprio semantic domain.

- project referent names;
- BA2 operator keys;
- operator-scoped role keys;
- semantic kind keys;
- controlled property keys;
- controlled value domains;
- local-structure keys quando formalmente ammessi.

Sinonimia, traduzione, naming similarity o LLM similarity non stabiliscono equivalenza normativa automaticamente.

Registry boundary

BA5 governa canonicalita' e resolution; non puo' ridefinire semantica, roleKey o cardinalita' di un operatore BA2. Per questi aspetti l'authority corrente resta BA2 R3.

35 BA4 - projection boundary

CURRENT / PRESERVED

governed documentation
 -> accepted BA
 -> projection

-> method-owned interpretation

Ogni meaning-bearing projection item deve tracciare alla BA.

35.1 Coverage modes

EXHAUSTIVE_FOR_DECLARED_SCOPE
SELECTIVE

Una selective projection puo' omettere elementi senza dichiararli falsi o assenti. Una method-owned interpretation puo' aggiungere semantica del metodo downstream, ma deve restare trace-bound e non diventare BA/project authority.

35.2 Projection non autorizza inference upstream

Un DFD, un graph, una threat taxonomy o una vista architetturale non possono costringere la BA ad aggiungere endpoint, trust boundary, actor, data flow o service relationship non governati.

36 Regression e acceptance della Base Analysis

CURRENT / PRESERVED

Dopo una material BA change o contract refinement:

1. ricostruire o aggiornare la BA contro la source pin;
2. verificare ogni proposition materiale;
3. verificare diagnostics e forbidden inference;
4. controllare property/constraint scope;
5. controllare request/case/identity/evaluation bindings;
6. verificare signatures e cardinalita';
7. rieseguire usefulness/projection checks appropriati;
8. cercare counterexample e semantic loss;
9. riaprire soltanto il minimo owning layer se necessario.

36.1 Checklist pre-acceptance

1. authority e source revision sono pinned?
2. ogni referent ha identity solo quando necessaria?
3. ogni proposition usa operator/role canonici?
4. tutte le cardinalita' required sono source-grounded?
5. ruoli optional sono presenti solo se governati?
6. e' stato introdotto un fatto non governato?
7. un NOT SPECIFIED e' diventato default?

8. provenance e revalidation sono sufficienti?
9. local structure candidate e' stata scambiata per normative vocabulary?
10. una candidate R25 e' stata presentata come admitted?
11. una projection puo' consumare la BA senza reinterpretare prose per shared meaning gia' rappresentato?
12. esiste un counterexample che richiede targeted reopen?

37 Open register R25 da preservare nella prossima revisione

OPEN / COUNTEREXAMPLE-GATED

ID	Tema	Stato corrente
PR-01	perform / execution identity	PRESSURE RETAINED; irreducibility not demonstrated.
PR-02	pipeline composition/order	OPEN.
PR-03	interface/path/invoke	RECONCILED / CC-04.
PR-04	boundary/interaction	OPEN.
PR-05	ordered comparison	ROUTE CL-01.
PR-06	scalar/property comparison	ROUTE CL-01.
PR-07	structured information contract	OPEN.
PR-08	at-rest storage	RECONCILED / CC-03.
PR-09	acquisition/refresh	OPEN.
PR-10	conditional action trigger	RECONCILED / CC-04 + CONDITION.
PR-11	data/evidence consumption	RECONCILED.
PR-12	negative implication/non-sufficiency	OPEN.
PR-13	bounded/ranked/recency selection	REUSABLE SELECTION STRUCTURE / NON-NORMATIVE.
PR-14	applicability/configuration binding	OPEN.
CMD-OP04	remove observe.result	DEFERRED.
CMD-OP05	transition state/value refinement	DEFERRED.
OBS-OT-01	operation-target/effect-scope	LEVEL UNRESOLVED.

38 Cosa questa revisione non chiude

Questa candidate non chiude:

- BA6 integrated acceptance;
- formal admission di provideService, storedIn, initiate;
- ordered/scalar comparator vocabulary oltre equals/notEquals;
- typing finale della selection structure;
- pipeline membership/order representation;
- function/process/behavior identity pressure;

- boundary/interactions, structured information, acquisition/refresh, negative non-sufficiency, applicability binding;
- CMD-OP04, CMD-OP05 e OBS-OT-01.

39 Gate di review della candidate example-enriched

R25 EVIDENCE

La review di questa candidate deve verificare, senza assumere che la presenza di un esempio chiuda automaticamente la semantica:

1. ogni top-level operator conserva definizione, firma, roleKey e cardinalita' della R2/R3 current authority;
2. ogni esempio e' source-grounded oppure marcato esplicitamente come didattico non normativo;
3. Facial Access e DermaTriage vengono usati come evidence, non come fonte automatica di nuove regole;
4. il percorso source meaning → semantic facts → BAREferent → BAProposition e' ricostruibile;
5. rejected alternative routing e forbidden inference restano visibili;
6. condition language, selection, pipeline/order e candidate construct mantengono lo status epistemico corretto;
7. i casi senza clean positive control non vengono artificialmente forzati.

Acceptance criterion della candidate example-enriched

La guida potra' essere considerata promotion-eligible soltanto dopo una review cumulativa R3 → R4 che dimostri che nessuna distinzione stabile e' andata persa e che nessun esempio ha promosso implicitamente un costrutto OPEN o CANDIDATE.

A Quick syntax sheet - top-level operators

```
transfer
  behavior    -> BAREferent [0..1]
  source      -> BAREferent [1]
  destination -> BAREferent [1..*]
  content     -> BAREferent [1..*]

produce
  actor  -> BAREferent [1]
  result -> BAREferent [1..*]
  input  -> BAREferent [0..*]

create
  actor    -> BAREferent [1]
  created  -> BAREferent [1..*]

observe
  actor    -> BAREferent [1]
  observed -> BAREferent [1..*]
```

```

    result    -> BReferent [0..*]

transition
  subject    -> BReferent [1]
  toState    -> BReferent [1]
  actor      -> BReferent [0..1]
  fromState  -> BReferent [0..1]

correlate
  correlatedItem    -> BReferent [1..*]
  correlationContext -> BReferent [1]

reference
  referencer -> BReferent [1]
  referenced -> BReferent [1..*]

dependOn
  dependent    -> BReferent [1]
  prerequisite -> BReferent [1..*]

consumeService
  consumer -> BReferent [1..*]
  service  -> BReferent [1]
  provider -> BReferent [0..1]

realize
  abstract    -> BReferent [1]
  realization -> BReferent [1..*]

assignResponsibility
  responsibleParty    -> BReferent [1..*]
  responsibilityScope -> BReferent [1]
  responsibilityKind  -> BReferent [1]

constrain
  constraintTarget -> BReferent [1]
  constraintValue  -> controlled value/structure [1..*]

classify
  classifiedReferent -> BReferent [1]
  semanticKind       -> BReferent [1..*]

decisionRule
  actor  -> BReferent [1]
  input  -> BReferent [1..*]
  result -> BReferent [1]
  rule   -> operator-local structure

```

B Quick syntax sheet - condition language

```

comparison
  referent      -> BReferent [1]
  property      -> controlled semantic key [1]
  comparisonKey -> equals | notEquals
  value         -> typed local value | BReferent [1]

```

```

satisfies
  subject    -> BReferent [1]
  condition -> BReferent [1]

allof
  childCondition -> decisionCondition [2..*]

anyOf
  childCondition -> decisionCondition [2..*]

not
  childCondition -> decisionCondition [1]

resultAssignment
  target -> BReferent [1]
  value  -> typed local value | BReferent [1]

```

C Quick status sheet - non-current constructs

```

selection
  REUSABLE_SELECTION_STRUCTURE / NON-NORMATIVE

provideService
  TESTED_POSITIVE / NOT_ADMITTED
  signature = NOT FROZEN

storedIn
  TESTED_POSITIVE / NOT_ADMITTED
  signature = NOT FROZEN

initiate
  TESTED_POSITIVE / NOT_ADMITTED
  working minimum:
    initiator -> BReferent [1]
    initiated -> BReferent [1]

pipeline/order
  OPEN
  no new top-level operator admitted

```

D Matrice degli accoppiamenti semantici ricorrenti

Questa matrice risponde a una domanda pratica frequente: *quando due ruoli o componenti “si accoppiano”, quale operatore descrive davvero la relazione?* La risposta non dipende dal nome tecnico degli endpoint ma dal significato governato.

Regola fondamentale

Un pattern architetturale o una coppia di ruoli non crea automaticamente un operatore. La stessa coppia `client/API`, `producer/consumer` o `provider/consumer` può richiedere proposition diverse a seconda che la source governi *initiation*, *service consumption*, *transfer*, *production*, *constraint*, *responsibility* o *correlation*.

Pattern intuitivo	Significato governato	Operator / composition	Come leggerlo e cosa non inferire
Provider – Service – Consumer	il consumer usa effettivamente una capability/service	consumeService; provider opzionale se governato	Il provider puo' essere incluso nel ruolo opzionale current. provideService e' candidate separato solo se la provider-service relation deve vivere indipendentemente dal consumer. Positive R25 evidence, ma non usare come operator current finche' G1–G8 non chiude l'admission.
Service Provider – Service	una parte fornisce un service indipendentemente da uno specifico consumer	provideService CANDIDATE / NOT ADMITTED	
Producer – Result	un actor rende disponibile un result	produce	Non implica destination, consumer, transfer o nuova identity.
Producer – Result – Consumer	A produce X; B usa X in una propria operazione	produce(A, X) + X come input nella proposition di B	Non esiste un generic consumeData a current: CC-01 e' stato respinto come ridondante nello scope testato.
Sender – Content – Receiver	conveyance esplicita source-to-destination	transfer	Payload, source e destination sono governati. Non implica service consumption, success o protocollo.
API / Endpoint – Client	il client avvia/invoca una capability	initiate CANDIDATE ; eventualmente consumeService se la source governa service use	“Client usa API” non basta: separare initiation, service consumption, request/response transfer e constraints di autenticazione.
API – Request/Response – Client	request/response content attraversa un boundary	transfer per i payload; correlate per same-request binding	Non modellare automaticamente l'endpoint come service provider o l'invocazione come transfer se la source non governa il payload.
Client – Protected API	l'uso e' consentito solo con un meccanismo/credential governato	constrain sul target o sul contract; altri operatori descrivono l'azione	Esempio DermaTriage: X-API-Key e bearer JWT. Constraint != authentication execution.
Initiator – Process/Operation	una parte avvia un processo/operazione	initiate CANDIDATE / NOT ADMITTED	Non implica che il processo completi con successo, produca risultato o consumi un service.
Observer – Existing State/Data	lettura/query/ispezione di significato esistente	observe	Se la query seleziona membri secondo criteri, la selected-membership semantics resta separata.
Selector – Candidate Population – Selected Result	scelta governata fra candidati	local selection structure NON-NORMATIVE	Ranking, bound, recency e selection sono distinzioni separabili. Nessun top-level operator nuovo ammesso.
Classifier – Referent – Kind	assegnazione di category/kind	classify	Non confondere con selection tra candidati, transition di lifecycle o generic property assignment.
Decision Actor – Inputs – Result	mapping governato da conditions/input a result	decisionRule + condition language	Un allowed result domain o una conditional action non bastano da soli.
Responsible Party – Scope – Responsibility Kind	authority/responsibility esplicita	assignResponsibility	Provider, actor, producer e owner tecnico non diventano responsible party per default.

Pattern intuitivo	Significato governato	Operator / composition	Come leggerlo e cosa non inferire
Abstract Capability – Concrete Realization	un elemento concreto materializza un significato astratto	realize	Non equivale a perform/execute e non assegna responsibility.
Dependent – Prerequisite	prerequisite semantico	dependOn	Temporal order, pipeline adjacency e “usa prima” non bastano.
Referencer – Referenced	A riguarda/punta a B senza binding piu’ forte	reference	Se serve same-case/same-request identity, usare correlate.
Target			
Item(s) – Same Context	same-request/case/evaluation binding	correlate	Non e’ generic association: il shared context deve essere materialmente governato.
Subject – From/To State	stessa identity attraversa state/lifecycle change	transition	Version selection, property update e rollback support non bastano senza same-identity state semantics.
Actor – New	nasce una nuova identity	create	Non confondere con output production o persistence.
Item/Occurrence	project-semantic		
Target –	restriction governata	constrain	Local condition, selection bound e decision mapping restano concetti diversi.
Restriction/Domain	riusabile/queryable		
Stored Item – Store	at-rest association	storedIn	Non assorbire in create, transfer o observe; non usare come current operator finche’ non ammesso.
	materialmente indipendente	CANDIDATE / NOT ADMITTED	precedes != dependOn != transfer != initiate. La pipeline non e’ un top-level operator current.
Pipeline Stage A – Stage B	membership/order fra stadi	OPEN local composition question	

E Matrice rapida: quale operatore provo per primo?

Domanda semantica	Primo costruito da valutare	Boundary da controllare subito
Chi rende disponibile un risultato?	produce	create, transfer, decisionRule
Nasce una nuova identity/occurrence?	create	produce, transition
Qualcosa passa da source a destination?	transfer	produce, consumeService, initiate candidate
Qualcuno legge/ispeziona qualcosa di esistente?	observe	selection, transfer, transition
La stessa identity cambia stato?	transition	version selection, qualification, property update
Due elementi devono restare nello stesso case/request?	correlate	reference
A riguarda B senza same-context requirement?	reference	correlate, dependOn
A richiede B come prerequisite?	dependOn	temporal order, pipeline order
A usa una capability/service?	consumeService	endpoint presence, transfer, provider relation
Un concreto materializza un abstract meaning?	realize	execution, classify, responsibility
La source assegna responsibility/authority?	assignResponsibility	actor/provider/producer roles
Esiste una restriction riusabile sul target?	constrain	local condition, decision mapping, selection bound
A appartiene a un semantic kind?	classify	selection, transition, property assignment
Conditions/input determinano un result?	decisionRule	conditional action, allowed domain, selection
La source governa "best/top-N/recent-N"?	local selection candidate structure	ranking, bound, observe/retrieval
La source governa "invoca/avvia"?	initiate candidate	transfer, consumeService, success/completion
La source governa provider-service indipendente?	provideService candidate	consumeService.provider optional role
La source governa at-rest association?	storedIn candidate	create, observe, transfer

F Quick question matrix - dalla domanda al costruito

Domanda rapida	Se SI, provare	Non confondere con
Viene reso disponibile un risultato?	produce	create, transfer
Nasce una nuova identity/occurrence?	create	produce, transition
Si legge/ispeziona qualcosa di esistente?	observe	transfer, selection
La stessa identity cambia stato?	transition	property update, classification
Content passa da source a destination?	transfer	consumeService
Serve same-context binding?	correlate	reference
A richiama semplicemente B?	reference	correlate, dependOn
B e' prerequisite reale di A?	dependOn	ordering
A usa un service/capability B?	consumeService	transfer, provider relation
Concrete X materializza abstract Y?	realize	execution/perform
Responsibility/authority e' assegnata?	assignResponsibility	actor/provider/producer
Un target e' limitato da valore/dominio?	constrain	condition locale, classify
Viene assegnato un semantic kind?	classify	state/value assignment
Conditions/input determinano result?	decisionRule	conditional action
X soddisfa condition Y senza property model?	satisfies	invented boolean comparison
Si sceglie subset/top-K/recent-N/best?	local selection	ranking, bound, observe
Si descrive membership/order di pipeline?	OPEN local structure review	dependOn, transfer
A avvia B senza implicare esito?	initiate CANDIDATE	transfer, create, transition
Nessuna forma preserva il significato?	diagnosis / pressure	local vocabulary invention

Ultima domanda prima di accettare una proposition

Se rileggo soltanto la source governata, posso giustificare ogni referent, roleKey, cardinalita' effettivamente popolata e condition della proposition senza usare conoscenza dell'implementazione, convenzioni del tool o inferenze downstream? Se NO, la proposition non e' ancora accettabile.

G Provenance degli esempi principali

Corpus / source	Uso nella guida
Facial Access candidate R2 - D-3.1, D-3.4, D-3.5, D-3.6, FR-3.2.1, FR-3.4.2	realize, assignResponsibility boundary, dependOn, consumeService, constrain, produce, transfer, correlate. Candidate documentation usata come evidence didattica; non viene promossa a project authority da questa guida.
Facial Access accepted BA evidence richiamata dalla R3 DermaTriage governed R1 - DEC-12..18 e FR-01..27	reference, decisionRule, role/signature regression e operator boundaries. produce, transfer, correlate, reference, dependOn, consumeService, constrain, classify, decisionRule, selection, pipeline/order, candidate construct pressure e negative controls.
R25 working findings / pressure ledger	status di CC-01..CC-04, PR-01..PR-14, CL-01..CL-05, CMD-OP04/05 e OBS-OT-01.